

AI-Powered Emergency Ambulance Dispatch & Routing System

Omni Disaster Management – Software Problem Statement 17: Reducing Ambulance Response Time in Cities

Objective: Reduce ambulance response time by selecting the ambulance with the lowest predicted response time, optimizing routes using changing road conditions, continuously updating ETA, and recommending suitable hospitals based on emergency requirements.

1. Problem Understanding

In traffic-heavy cities, ambulance response time can increase significantly because of congestion, road conditions, distance, inefficient ambulance allocation, and changing traffic conditions. Simply assigning the geographically nearest ambulance does not always produce the fastest response. A farther ambulance on a less congested route may reach the patient earlier.

The challenge continues after dispatch: traffic and road conditions can change during transit, while the destination hospital must have facilities appropriate for the patient's emergency. Therefore, the system connects emergency intake, structured triage, ambulance allocation, dynamic routing, ETA prediction, live tracking, and hospital recommendation into one workflow. **Project perspective:** Existing emergency infrastructure such as India's 112 Emergency Response Support System and 108 ambulance services already provides important emergency-response capabilities. This project therefore does not attempt to replace them. Instead, it proposes an AI and optimization layer that can improve ambulance selection, routing, ETA prediction, and hospital matching.

2. Proposed Solution

We propose an **AI-assisted Emergency Ambulance Dispatch & Routing System** that receives an emergency request, determines its location and priority, selects the ambulance with the lowest predicted response time, calculates the fastest practical route, continuously monitors the ambulance, and recommends a suitable hospital.

Core workflow: Emergency Request → GPS Location → AI-Assisted Triage → Ambulance Selection → Traffic-Aware Route → Live Rerouting → Hospital Matching → Live Monitoring → Emergency Completion

The dispatch engine considers ambulance availability, location, predicted travel time, emergency priority, vehicle suitability, and traffic conditions rather than distance alone. Map and traffic data provide dynamic travel costs, while A/Dijkstra calculates candidate routes. An ML model can predict ETA using historical and real-time trip features.

3. System Architecture

Layer	Components / Responsibility
Emergency Intake	Web/mobile request, emergency type, patient location, contact and basic incident information
AI Triage Dispatch	Uses structured emergency information to classify severity and assign response priority; AI provides decision support rather than medical diagnosis
Engine Routing	Ranks available ambulances using predicted response time, location, availability, emergency priority, and vehicle suitability.
Engine Real-Time	A*/Dijkstra + map/traffic data for fastest practical route
Layer	GPS updates through Socket.IO, live ETA updates, route-deviation detection, and dynamic rerouting based on changing road conditions.
Hospital Matching	Ranks hospitals using estimated arrival time, required facilities, emergency suitability, and available capacity where data is available.
Command Center	Live map, active incidents, ambulance status, ETAs, alerts and operational analytics
Data & Analytics	MongoDB stores incidents, routes, ambulance activity, hospital information and historical response data

Data flow: User/Control Centre → Backend API → Triage & Dispatch → Routing/Traffic Services → Selected Ambulance → GPS/Socket.IO updates → ETA & Rerouting → Hospital Recommendation → Command-Centre Dashboard.

4. Key Features

- **Intelligent ambulance allocation:** Selects the best ambulance based on availability, location, predicted arrival time and suitability rather than distance alone.
- **AI-assisted emergency prioritization:** Uses structured emergency information to classify severity and prioritize critical cases, with AI used as decision support rather than autonomous medical diagnosis.
- **Traffic-aware routing:** Uses road-network and traffic data to assign dynamic travel costs and calculate the fastest practical route.
- **Dynamic rerouting:** Recalculates the route when congestion, road blockage, delay or route deviation is detected.
- **Live ambulance tracking:** Displays GPS position, status and predicted ETA in real time.
- **Smart hospital matching:** Recommends hospitals based on travel time, required facilities and available capacity/bed information where available.
- **Command-centre dashboard:** Provides a single operational view of active emergencies, ambulances, routes, ETAs and alerts.
- **Historical analytics:** Identifies high-delay zones, accident hotspots, recurring congestion and ambulance response patterns.
- **Optional traffic-signal priority:** Can integrate with smart-city traffic infrastructure to request priority along the ambulance route.

5. Technology Stack

Area	Technology
Frontend	React.js + Vite, Tailwind CSS
Maps & GIS	Leaflet/Mapbox; OpenStreetMap + OSRM as a lower-cost alternative
Backend	Node.js + Express.js
Real-time communication	Socket.IO
Database	MongoDB / MongoDB Atlas
AI & Optimization	Python + FastAPI, A/Dijkstra for route optimization, and ML-based ETA/response-time prediction.
ML options	XGBoost or Random Forest for response-time prediction using features such as distance, traffic level, time of day, road conditions, and historical travel time.
Location	GPS / device location data
Deployment	Vercel for frontend; Render/Railway for backend; MongoDB Atlas for database

6. Implementation Plan

Phase 1 – Requirements & Data Model: Define emergency, ambulance, hospital, route and user entities. Identify required inputs for triage, ambulance status and hospital capabilities.

Phase 2 – Core Backend: Build REST APIs with Node.js/Express and MongoDB for emergency requests, ambulance availability, hospital records and incident history.

Phase 3 – Dispatch Engine: Implement ambulance ranking using availability, predicted ETA, distance, severity and vehicle suitability.

Phase 4 – Routing & Maps: Integrate mapping and routing services. Implement A*/Dijkstra and calculate route distance, travel time and alternative routes.

Phase 5 – Real-Time Tracking: Connect GPS updates through Socket.IO. Show ambulance movement, status and ETA on the command-centre map.

Phase 6 – AI Layer: Train an initial ETA/response-time model using historical or realistic simulated trip data. Evaluate prediction error and compare ML-based ETA with route-engine estimates.

Phase 7 – Hospital Matching: Create a hospital scoring mechanism using estimated arrival time, required facilities, emergency suitability, and available capacity where data is available.

Phase 8 – Dashboard & Alerts: Build command-centre views for incidents, ambulances, routes, priority levels, ETA and route-change alerts.

Phase 9 – Testing & Simulation: Simulate multiple emergencies, traffic changes, road blockages, and ambulance movements. Compare intelligent dispatch and dynamic routing against nearest-ambulance and static-route baselines using response time, ETA accuracy, allocation quality, and rerouting performance.

Phase 10 – Deployment & Demonstration: Deploy the prototype and demonstrate an end-to-end emergency scenario from request creation to ambulance dispatch, live rerouting and hospital recommendation.

Expected Outcome

The prototype will compare intelligent dispatch and dynamic routing against nearest-ambulance and static-route baselines. Evaluation metrics will include average response time, dispatch-to-arrival time, ETA prediction error, route-recalculation time, critical-case prioritization rate, and successful hospital matching.

Key USP: “The system doesn't simply select the nearest ambulance; it selects the ambulance with the lowest predicted response time, dynamically reroutes it using changing road conditions, and recommends a suitable hospital based on emergency requirements and available facilities.”